



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Design with classes and Interface

Instructor: Le Thi Ngoc Hanh, Ph.D

ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

Benefits of Polymorphism

- ☾ **Flexibility:** allow objects to behave as if they belong to different classes, making the code more adaptable.
- ☾ **Reusability:** facilitate code reuse by enabling a single interface or superclass method to be implemented in multiple ways by various subclasses.
- ☾ **Maintainability and Scalability:** Systems that use polymorphism require fewer code changes when new classes are added or existing ones are modified.
- ☾ **Understandability:**
By enabling modular development, polymorphism makes the code easier to understand.

Lecture Objectives

A deeper look into
Inheritance & Polymorphism
stories

Abstract Classes and Methods

- ✧ An abstract class is a class **declared** using the keyword `abstract`.
- ✧ It is used when we want to define a general concept but not a complete implementation.
- ✧ Abstract classes are typically used as superclasses in an inheritance hierarchy.
- ✧ They cannot be instantiated because they are incomplete.
- ✧ They provide a common design and structure for subclasses.

Concrete Classes and Methods

- Classes that can be used to instantiate objects are called concrete classes.
- Such classes provide implementations of every method they declare (some of the implementations can be inherited).
- Abstract superclasses are too general to create real objects - they specify only what is common among subclasses.
- Concrete classes provide the specifics that make it reasonable to instantiate objects.
- Not all hierarchies contain abstract classes.

Abstract vs Concrete Example

```
class Employee {  
    protected String name;  
    public Employee(String name) {  
        this.name = name;  
    }  
    public double calculateSalary() {  
        return 0;  
    }  
}
```



```
Employee e = new Employee("John");  
System.out.println(e.calculateSalary());
```

Abstract vs Concrete Example

★ Assuming that new class is derived.

```
class PartTimeEmployee extends Employee {  
    public PartTimeEmployee(String name) {  
        super(name);  
    }  
}
```

```
PartTimeEmployee p = new PartTimeEmployee("Alex");  
System.out.println(p.calculateSalary());
```

Problem!!!

Before

```
class Employee {  
    protected String name;  
    public Employee(String name) {  
        this.name = name;  
    }  
    public double calculateSalary() {  
        return 0;  
    }  
}
```

After

```
abstract class Employee {  
    protected String name;  
  
    public Employee(String name) {  
        this.name = name;  
    }  
  
    public abstract double calculateSalary();  
}
```

```
class FullTimeEmployee extends Employee {  
    private double monthlySalary;  
  
    public FullTimeEmployee(String name, double salary) {  
        super(name);  
        this.monthlySalary = salary;  
    }  
  
    @Override  
    public double calculateSalary() {  
        return monthlySalary;  
    }  
}
```

```
class PartTimeEmployee extends Employee {  
    private int hours;  
    private double rate;  
  
    public PartTimeEmployee(String name, int hours, double rate) {  
        super(name);  
        this.hours = hours;  
        this.rate = rate;  
    }  
  
    @Override  
    public double calculateSalary() {  
        return hours * rate;  
    }  
}
```

Abstract vs Concrete Example

★ Polymorphism at run-time:

```
public class Demo_Employee {  
    public static void main(String[] args) {  
        Employee e1 = new FullTimeEmployee("John", 1000);  
        Employee e2 = new PartTimeEmployee("Alex", 20, 10);  
  
        System.out.println(e1.calculateSalary());  
        System.out.println(e2.calculateSalary());  
    }  
}
```

overriding vs overloading vs polymorphism

```
class Animal {  
    void sound() {  
        System.out.println("Animal");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog");  
    }  
  
    void sound(String mood) {  
        System.out.println(mood + " Dog");  
    }  
}
```

```
Dog d = new Dog();  
d.sound();           // overriding  
d.sound("happy");    // overloading  
  
Animal a = new Dog();  
a.sound();           // polymorphism
```

More Example

★ Without polymorphism

```
Dog d1 = new Dog();
```

```
Cat d2 = new Cat();
```

```
d1.sound();
```

```
d2.sound();
```

★ With polymorphism

```
Animal[] animals = {
```

```
    new Dog(),
```

```
    new Cat()
```

```
};
```

```
for (Animal a : animals) {
```

```
    a.sound();
```

```
}
```

Supertype and subtype

- ✧ A supertype (parent class) represents a general abstraction that **captures** common attributes and behaviors shared across multiple related entities.
- ✧ A supertype defines shared structure (state) and shared behavior (methods); it enables code reuse and conceptual modeling of “is-a” relationships.
- ✧ A supertype is often:
 - ▶ Concrete class (can be instantiated), or
 - ▶ Abstract class (cannot be instantiated, used for modeling)
 - ▶ Interface modeling

Supertype and subtype

- ★ A subtype (child class) is a class that Inherits from a superclass (extends), OR Implements an interface (implements)
- ★ Maintains is-a relationship

Example: FullTimeEmployee is-a Employee

PartTimeEmployee is-a Employee

Subtype provides HOW behavior is implemented

Sample contexts

```
interface Payment {  
    void pay();  
}
```

```
class CreditCard implements Payment {  
    public void pay() {  
        System.out.println("Credit card payment");  
    }  
}
```

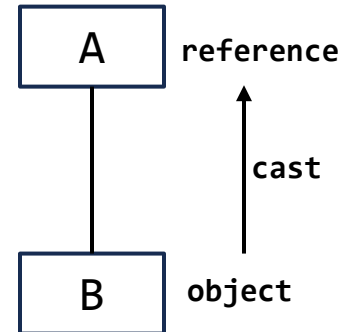
```
class PremiumZaloPay extends ZaloPay {  
    public void pay() {  
        System.out.println("Premium ZaloPay payment");  
    }  
}
```

```
class Employee {  
    protected String name;  
}  
  
class FullTimeEmployee extends Employee {  
    private double monthlySalary;  
}
```

Upcasting

Upcasting is the process of converting a subtype object into a supertype reference.

- ▶ Occurs automatically (implicit casting)
- ▶ No explicit cast required
- ▶ Always safe (no runtime risk)
- ▶ Enables polymorphism



Upcasting - Example

```
FullTimeEmployee f = new FullTimeEmployee(...);  
PartTimeEmployee p = new PartTimeEmployee(...);  
  
double total = f.calculateSalary() + p.calculateSalary();
```



```
Employee e1 = new FullTimeEmployee(...);  
Employee e2 = new PartTimeEmployee(...);  
  
double total = e1.calculateSalary() + e2.calculateSalary();
```

Downcasting and instanceof

Downcasting is the process of converting a supertype reference back into a subtype reference.

- Requires explicit cast
- May be unsafe (runtime error possible)
- Used when we need to access subtype-specific behavior

upcasting →

```
Employee e = new FullTimeEmployee(...);
```

downcasting →

```
FullTimeEmployee f = (FullTimeEmployee) e;
```

Invalid
downcasting →

```
Employee e = new PartTimeEmployee(...);  
FullTimeEmployee f = (FullTimeEmployee) e;
```

Downcasting and instanceof

- Solution: Use `instanceof` to verify before casting:

```
if (e instanceof FullTimeEmployee) {  
    FullTimeEmployee f = (FullTimeEmployee) e;  
    System.out.println(f.calculateSalary());  
}
```

- This solution can prevent runtime errors and ensures safe downcasting.
- Good OOP design reduces need for downcasting:

```
if (e instanceof FullTimeEmployee) { ... }
```

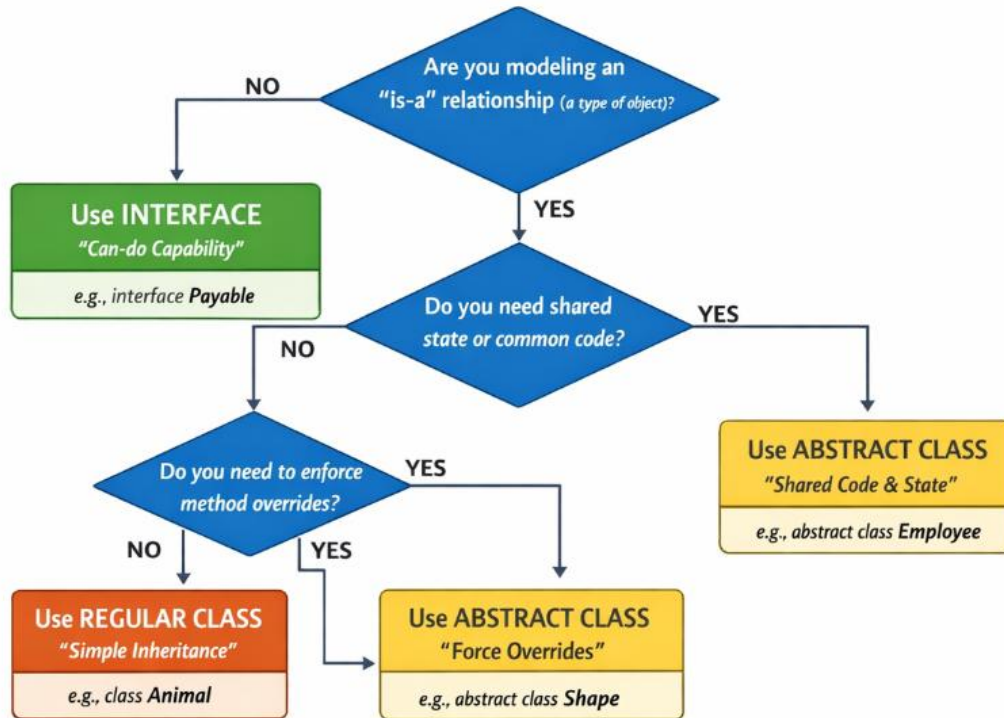


```
e.calculateSalary(); // polymorphism
```

Abstract Class vs. Interface

- An abstract class is used when we want to:
 - Represent a base concept with shared state and behavior
 - Provide partial implementation
 - Enforce some methods via abstract methods
 - Can have instance variables (state), constructors, concrete methods (implemented), abstract methods (unimplemented)
 - Supports single inheritance only
 - Models “is-a” relationship with shared structure
- An interface defines a pure behavioral contract:
 - Specifies what must be done, not how
 - No object state (except constants)
 - Methods are implicitly **public**
abstract; fields are **public**
static final
 - Supports multiple implementation
 - Models capability / role-based design

Abstract Class, Interface, or Regular Class?



Boundary

- ☾ In OOP, inheritance provides flexibility, but unrestricted extension can:
- ▶ Break intended behavior
 - ▶ Violate security or business rules
 - ▶ Introduce incorrect overriding
 - ▶ Java uses the final keyword to define boundaries (restrictions)

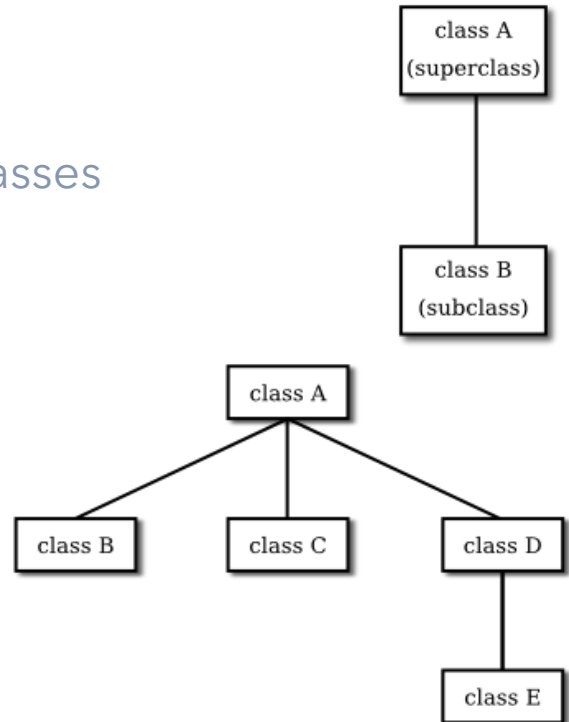
Boundary – final Class

★ A final class:

- ▶ Cannot be extended
- ▶ Prevents creation of subclasses

★ Syntax:

```
final class ClassName {  
    // implementation  
}
```



Boundary – final Method

★ A final method:

- ▶ Can be inherited
- ▶ But cannot be overridden by subclasses

```
class Employee {  
    public final void displayInfo() {  
        System.out.println("Employee Info");  
    }  
}
```

Boundary - final Variable

A **final** variable:

- ▶ Can be assigned only once
- ▶ Value cannot be changed after initialization
- ▶ A final variable must be initialized at declaration, or inside constructor (for instance variables)

```
final double TAX_RATE = 0.1; ← constant
```

```
class Employee {  
    final String id;  
  
    public Employee(String id) {  
        this.id = id; // allowed (assigned once)  
    }  
}
```